Trading Bot PRD (v0.1)
Date: 2026-10-17
---
1. Overview and Goals
The bot takes your own **BTC/USDT perpetual** trades on **Bybit** based on written rules, so emotio
cannot change an entry, a stop or an exit. v1 targets Bybit and one user on a small account, and
every strategy, exit policy and exchange is a **swappable module**.
- **Take every valid setup of Strategy■1 and nothing else**, with no manual override once a trade is
open.
- **Enforce risk in code**: at most 2 trades per day, risk per trade from the capital schedule, and
a pre■trade gate on fees and slippage.
- **Pluggable architecture**: strategies, exit policies and exchange adapters are separate modules.
- **Run the same code in backtest, demo and live**.
- **Run unattended on a free server**, and report what it did and why it skipped setups.
Success means every logged entry, stop and exit traces to a written rule, no trade exceeds its risk
budget or daily cap, and every rejected setup is logged with the rule that rejected it.
---
2. Scope and Assumptions
- **In■scope**: BTC/USDT perpetual on Bybit, longs & shorts, one user. Strategy■1, three exit■policy
variants, bot wallet with 24■hour withdrawal lock. Backtest, Bybit demo and small live modes, alerts
and daily report.
- **Not in v1**: other symbols, DEX adapters, pooled multi■user funds, ML, UI beyond alerts and
reports.
- **Assumptions**:
  * 1■pt = $1 per 1■BTC.
  * All times stored in UTC. Trading day starts at 02:00■Lagos (01:00■UTC).
  * One open position at a time, isolated margin, stop & TP placed on exchange.
---
3. Architecture
```mermaid
flowchart LR
    subgraph Core[Core Engine]
        Strategy[Strategy Plugin]
        RiskEngine[Risk Engine]
        ExitPolicy[Exit Policy Plugin]
        StateStore[State Store (SQLite)]
        Scheduler[Scheduler]
    end
    subgraph Exchange[Exchange Adapter]
        BybitAdapter[BybitAdapter]
        FakeAdapter[FakeAdapter (Backtest)]
    end
    subgraph UI[Admin UI]
        FastAPI[FastAPI + Jinja2]
        Auth[Auth (email/password)]
    end
    subgraph Alerts[Alerting]
        TelegramBot[Telegram Bot]
        EmailSender[SMTP Email]
    end
    Strategy -->|signal| RiskEngine
    RiskEngine -->|order request| BybitAdapter
    ExitPolicy -->|manage| BybitAdapter
    BybitAdapter -->|updates| StateStore
    StateStore -->|feeds| FastAPI
```

```
    FastAPI -->|commands| BybitAdapter
    FastAPI -->|alerts| Alerts
    Scheduler -->|timers| Core
    classDef external fill:#f9f,stroke:#333,stroke-width:1px;
    class BybitAdapter,FakeAdapter external;
```

3.1 Pluggable Components

Component	Interface (Python ABC)	Example Implementations
Strategy	`StrategyPlugin.generate_signal(candles) -> Signal	Strategy1, StrategyX
Exit Policy	`ExitPolicy.handle(position, market) -> actions`	DefaultExit, VariantA■D
Exchange Adapter	`ExchangeAdapter.place_order(...)`, `...get_position()`	`BybitAdapter`, `FakeAdapter`
Pair Component	`PairConfig(symbol, lot, tick, min_notional)`	BTCUSDT, ETHUSDT, …

---

4. Trade Management & Exit Policy (Default)

- **Stop**: sweep low minus configurable buffer (10■20■pt), reduce■only.
- **TP1**: at 3R, close ceil(0.7■×■qty/lot) lots.
- **TP2**: remaining lots at 5R.
- **Early exit**: armed at 1R, market close if price gives back 50■% of peak gain, until 1.5R.
- **Breakeven**: at 1.5R move stop to entry■+■entry fee■+■estimated exit fee■+■slippage buffer.
- **Sweep failure**: if the 15■m candle closes below the swing after entry, close at market (config flag).

Four back■test variants (A■D) modify breakeven/early■exit/TP target as described in the original PRD.

---

5. Risk Engine

5.1 Capital Schedule & Risk per Trade

Capital C	Risk per trade R
Below $50	$1
$50■$449	$1■+■$0.5■×■■(C■50)/50■
$450■$599	$5 (cap)
$600+	$5■+■$1■×■■(C■500)/100■

5.2 Leverage Ceiling

Capital	Leverage ceiling
<■$10	Disabled (notification)
$10■$50	Up to 100×
>■$50	Up to 50×

Leverage used is the smallest whole number fitting the **margin cap** (50■% of equity), never exceeding ceiling or Bybit limits.

5.3 Position Sizing

```
L = d + s_in + s_out + r_taker·(E + S - s_out)
qty = floor(R_trade / (L·lot)) * lot
```

- `E` best ask, `S` stop price, `d = E■S` (points).
- `s_in`, `s_out` estimated slippage (points).
- `r_taker` taker fee rate.
- `lot` quantity step.

5.4 Net RR Gate

```
net_RR = (3·d - w) / L  ≥  2.0   (configurable)
```

`w` = entry fee + entry slippage + maker exit fee per BTC.
5.5 Gate Order (first failure rejects setup)
1. Trading enabled for capital tier.
2. Daily cap not reached.
3. No open position.
4. Trend table allows a trade.
5. Quantity ≥ 1■lot & notional ≥ exchange minimum.
6. Net RR ≥ 2.0.
7. Leverage within ceiling & margin ≤■50■% equity.
8. Liquidation price ≥■3■×■stop distance.
---
6. Bot Wallet & Withdrawals
- Ledger on top of exchange account.
- Deposit → credit ledger.
- Entry: move required margin from funding wallet to futures wallet (isolated).
- Exit: margin + P/L returned to funding wallet.
- Withdrawal: request → 24■h timer → USDT sent to pre■registered address.
- API■key trade■only; optional whitelist; bot enforces lock.
---
7. Exchange Adapter & Bybit Details
- **Methods**: `get_candles`, `get_order_book`, `get_instrument_spec`, `get_fee_rates`, `place_order`, `amend_order`, `cancel_order`, `get_position`, `get_balance`, `set_leverage_and_margin_mode`, `transfer_to_trading`, `transfer_to_funding`, optional `withdraw`.
- **Bybit specifics** (BTCUSDT perpetual):
  - Tick size: 0.1
  - Minimum order: 0.001■BTC or 100■USDT notional
  - Fees (non■VIP): 0.055■% taker, 0.02■% maker
  - Demo endpoint: `api-demo.bybit.com` (REST for orders).
---
8. Failure Handling & Hosting
- **Crash / restart**: reload open positions, orders, balances, daily count; resume management.
- **Duplicate protection**: client order ID derived from setup; restart cannot resend.
- **API errors**: retry with exponential back■off, verify state before retry.
- **Authentication failure**: stop opening trades, keep managing open ones, send alert.
- **Kill switch**: immediate close of open position & disable new entries.
- **Hosting**: lightweight container (Docker) on a free■tier host with stable outbound IP (Fly.io). Persistent volume for SQLite DB, env■vars for secrets, health■check endpoint.
- **IP whitelisting**: detect expired keys, alert user.
---
9. Monitoring & Reporting
9.1 Alerts (Telegram & Email)
- Bot start/stop, API/auth failures.
- Trade opened/closed with result in R.
- Setup rejected (once per setup) with rejecting rule.
- Capital below $10 → trading disabled.
- Withdrawal request / release, kill■switch usage.
9.2 Rejection Funnel
Stages recorded: leg too small, pullback too shallow, no sweep, weekly & daily range, net RR gate, size too small, daily cap. Weekly report aggregates counts.
9.3 Daily Report (2■am Lagos start)
- Trades taken, R results, fees & slippage vs estimates, equity, capital tier, day's risk R.
- Per■trade expected vs actual fill price for slippage correction.
---
10. Modes & Rollout
| Mode | Data | Orders | Purpose |

|---|---|---|
Backtest	Historical 1■h, 15■m, 2■m candles	Fake exchange (fees & slippage model)	Rule correctness, rejection funnel, exit variants
Demo	Bybit mainnet prices	Bybit demo account (simulated funds)	Validate live behaviour, order handling, slippage logging
Live (small)	Mainnet	Real orders (minimum size)	Final check before real risk

Transition criteria:
- All tests green, funnel reviewed.
- Demo: ≥■20 clean trades (no rule deviations) over ≥■24■h.
- Live: after demo success, enable live mode.

---

11. Test■Driven Development Plan
1. **Pure functions**: risk schedule, position sizing, net■RR gate, daily■cap reset.
2. **Unit tests** for each pure module.
3. **Integration tests** with `FakeAdapter` for signal → order flow and exit■policy variants.
4. **End■to■end tests**: FastAPI + fake adapter, restart resilience, duplicate■order protection.
5. **Coverage** ≥■95■% (pytest■cov). CI stub prepared but **not run** until approval.

---

12. Open Questions (answered by user)
- **Programming language**: Python with asyncio (recommended).
- **Alert channel**: Both Telegram and Email.
- **Hosting**: Fly.io for stable IP.
- **Risk■percentage guard**: Not added (keep fixed $1 rule only).
- **Demo trades before live**: At least 20 clean demo trades.
- **Flexible pair component**: Implemented via `PairConfig` allowing additional symbols.

---

13. Next Steps (awaiting user "start building" confirmation)
1. Create project skeleton (`src/` package, Dockerfile, `fly.toml`).
2. Implement abstract interfaces and minimal Strategy■1 plugin.
3. Wire FastAPI admin UI (login, API■key screens, dashboard).
4. Add risk■engine gate logic.
5. Write unit■tests as per the TDD plan.
6. Implement `FakeAdapter` then `BybitAdapter`.
7. Configure alerts and daily report scheduler.
8. Deploy to Fly.io, run 24■h demo; after ≥■20 clean trades enable live mode.

Prepared by Antigravity (AGY). All diagrams are in Mermaid format and can be rendered in the IDE.